DigiNiwas Services CMS + Property Reviews — Postman Guide

Base URL and variables

Create a Postman environment with:

• baseUrl: http://localhost:5000/api/v1 (use your deployed backend origin followed by /api/v1 in staging/production)

• buyerToken: JWT access token for a signed-in buyer

• partnerToken: JWT access token for a signed-in partner

• sellerToken: JWT access token for a signed-in seller

• adminToken: JWT access token for an admin

• propertyMongoId: 6ab21a1b2213e3539c117479 (sample property _id; use a real existing NewProperty _id from your own database)

• reviewId: set from a review response's review._id

Do not put account passwords, OTPs, or stored password hashes into Postman collection variables or shared documentation. Obtain JWTs by using each role's normal login flow. Keep tokens in a local Postman environment, not a shared collection export.

1. Public Services CMS

Read public services content

• **Method:** GET

• **URL:** {{baseUrl}}/cms/services/public

• **Auth:** None

• **Purpose:** Gets the service cards and detail content. FAQ is an optional faqs array; if it is empty or missing, the service detail page hides the FAQ section.

Read Services CMS as admin

• **Method:** GET

• **URL:** {{baseUrl}}/cms/services/admin

• **Auth:** Bearer Token {{adminToken}}

Upload a service image (admin)

• **Method:** POST

• **URL:** {{baseUrl}}/cms/services/upload-image

• **Auth:** Bearer Token {{adminToken}}

• **Body:** form-data, key image with type **File** (JPG, JPEG, PNG, WEBP or AVIF; up to 20 MB).

• **Response:** { "success": true, "url": "https://..." }. Copy the returned URL into heroImage, an imageTags entry, or a detailSections[].image field, then save the Services CMS.

Update Services CMS as admin

• **Method:** PUT

• **URL:** {{baseUrl}}/cms/services/admin

• **Auth:** Bearer Token {{adminToken}}

• **Body:** raw JSON. Send the full services array when updating, since this endpoint replaces the stored array.

Example request body:

```
{
  "eyebrow": "EVERYTHING YOU NEED",
  "heading": "Everything You Need.",
  "subheading": "All Under One Roof.",
  "description": "Trusted support for every step of your property journey.",
  "services": [
    {
      "title": "Home Loan Assistance",
      "slug": "home-loan-assistance",
      "shortDescription": "Get guidance through the home loan journey.",
      "description": "Long-form service overview goes here.",
      "heroImage": "https://example.com/service-hero.jpg",
      "imageTags": ["https://example.com/gallery-1.jpg", "https://example.com/gallery-2.jpg"],
      "icon": "wallet",
      "features": ["Eligibility guidance", "Document checklist"],
      "process": ["Share requirements", "Review options", "Proceed with lender"],
      "benefits": ["Clear next steps", "Guided documentation"],
      "detailSections": [
        {
          "icon": "check-circle",
          "number": "01",
          "title": "Initial consultation",
          "content": "Main content for this step.",
          "subcontent": "Additional supporting content.",
          "image": "https://example.com/step-1.jpg"
        }
      ],
      "faqs": [
        { "question": "What documents do I need?", "answer": "The team will share a checklist based on your case." }
      ],
      "ctaText": "Enquire About Home Loans",
      "ctaLink": "/contact",
      "active": true
    }
  ]
}
```

detailSections supports icon, number, title, content, subcontent, and image. imageTags is a list of gallery image URLs. FAQs are rendered only when at least one valid question/answer pair is saved.

2. Property Reviews API

Reviews attach to the NewProperty MongoDB document. Use the property's MongoDB _id for propertyId in a review request; the human-readable property code such as DWR-5698779579 is not accepted for review creation.

List reviews for a property (public)

• **Method:** GET

• **URL:** {{baseUrl}}/property-reviews/property/{{propertyMongoId}}

• **Auth:** None

• **Response:** success, count, averageRating, reviews.

Create a property review

• **Method:** POST

• **URL:** {{baseUrl}}/property-reviews

• **Auth:** Bearer Token. Use {{buyerToken}}, {{partnerToken}}, or {{sellerToken}}.

• **Body:** raw JSON

```
{
  "propertyId": "{{propertyMongoId}}",
  "rating": 5,
  "remark": "The property information was clear and the visit was helpful."
}
```

Rules: rating must be an integer from 1 to 5; remark is required and limited to 2,000 characters; each account can submit one review per property. A second review from the same account for the same property returns HTTP 409. Reviewers must be authenticated as a buyer, partner, or seller. Review creation stores the authenticated account's name and role rather than accepting reviewer identity from the request body.

List all reviews in CMS (admin only)

• **Method:** GET

• **URL:** {{baseUrl}}/property-reviews/admin

• **Auth:** Bearer Token {{adminToken}}

Edit a review in CMS (admin only)

• **Method:** PATCH

• **URL:** {{baseUrl}}/property-reviews/admin/{{reviewId}}

• **Auth:** Bearer Token {{adminToken}}

• **Body:** send any fields to update

```
{
  "rating": 4,
  "remark": "Updated review text after moderation.",
  "isVisible": true
}
```

Set isVisible to false to hide the review from public property pages without deleting it.

Delete a review in CMS (admin only)

• **Method:** DELETE

• **URL:** {{baseUrl}}/property-reviews/admin/{{reviewId}}

• **Auth:** Bearer Token {{adminToken}}

• **Body:** none

Suggested test sequence

1. Call the public services GET request without a token; expect 200.

2. Call the admin services GET request without a token; expect 401; repeat with admin token; expect 200.

3. Call property reviews GET with propertyMongoId; expect 200 and an empty array if no reviews exist.

4. POST a review using a buyer token; expect 201. Repeat the same POST with the same buyer and property; expect 409.

5. Repeat review creation with a partner token and a seller token; each account can create one review for that property.

6. Use admin token to GET all reviews, PATCH one review, hide it (isVisible: false) and confirm it no longer appears in the public GET response; then test DELETE with a disposable test review.

7. Update service CMS with a FAQ and confirm it appears on that service detail page. Save faqs: [] and confirm the FAQ section disappears.

Common status codes

• 200: successful GET/update/delete operation.

• 201: review created.

• 400: invalid property MongoDB ID, invalid rating, or missing remark.

• 401: missing/invalid JWT for a protected route.

• 403: role not allowed or admin token required.

• 404: property/review not found.

• 409: this account has already reviewed this property.

Notes

• Use a valid NewProperty document ID from the current database; the sample ID above may not exist in another environment.

• If the API uses a different local port or deployed origin, change baseUrl.

• The backend's existing apiAccessControl middleware must allow the public services and property-review GET endpoints; this update adds those public GET rules.

• Admin endpoints require a JWT whose decoded role is admin.